Exercise 1

Computer Vision, Fall 2025

Name: *Chiara Zara*

Instructions:

- Return the answer in PDF and Jupyter Notebook formats.
- Return latest on Sunday 14.9 at 23.50 via Moodle.

Ex 1.1 Homogeneous Coordinates and Transformations (2 Points)

a) Translate (t), scale (s), and rotate (r) a point $c = (c_x, c_y)$ on the xy-plane to a new location (d_x, d_y) as illustrated below.

title

Using the translation $(2, 3)$, scaling $(-2, 8)$, and rotation (70°), answer the following:

(i) Provide the matrix representation for the translation using homogeneous coordinates.

(ii) Provide the matrix representation for the scaling using homogeneous coordinates.

(iii) Provide the matrix representation for the rotation using homogeneous coordinates.

(iv) Transform the point $c = (3, 4)$ on the xy-plane to a new location (d_x, d_y) using the translation and scaling operations given in sections (i) and (ii). Perform two separate tasks: first translation then scaling, and first scaling then translation. Compare the final locations and state if the order of transformations matters. You only need to provide the final locations.

Hint:

- We have provided empty matrices for you. Fill them with the correct values.

1.1.a i) Translation

```
Answer:
```

$$Translation: \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_x \\ c_y \\ 1 \end{bmatrix} = \begin{bmatrix} c_x + 2 \\ c_y + 3 \\ 1 \end{bmatrix}$$

1.1.a ii) Scaling

```
Answer:
```

$$Scale: \begin{bmatrix} -2 & 0 & 0 \\ 0 & 8 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_x \\ c_y \\ 1 \end{bmatrix} = \begin{bmatrix} -2c_x \\ 8c_y \\ 1 \end{bmatrix}$$

1.1.a iii) Rotation

```
Answer:
```

$$Rotation: \begin{bmatrix} cos70^\circ & -sin70^\circ & 0 \\ sin70^\circ & cos70^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_x \\ c_y \\ 1 \end{bmatrix} = \begin{bmatrix} 0.342c_x - 0.940c_y \\ 0.940c_x + 0.342c_y \\ 1 \end{bmatrix}$$

1.1.a iv) Comparing the Outcomes: Translation Followed by Scaling vs. Scaling Followed by Translation

```
Answer:
```

1) Translation-Scaling

-Translation:

$$T \cdot c = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 7 \\ 1 \end{bmatrix}$$

-Scaling:

$$S \cdot \begin{bmatrix} 5 \\ 7 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 & 0 & 0 \\ 0 & 8 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 5 \\ 7 \\ 1 \end{bmatrix} = \begin{bmatrix} -10 \\ 56 \\ 1 \end{bmatrix}$$

2) Scaling-Translation

-Scaling:

$$S \cdot c = \begin{bmatrix} -2 & 0 & 0 \\ 0 & 8 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \\ 1 \end{bmatrix} = \begin{bmatrix} -6 \\ 32 \\ 1 \end{bmatrix}$$

-Translation:

$$T \cdot \begin{bmatrix} -6 \\ 32 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -6 \\ 32 \\ 1 \end{bmatrix} = \begin{bmatrix} -4 \\ 35 \\ 1 \end{bmatrix}$$

Comparison

- Translation then Scaling $d_1 = (-10, 56)$
- Scaling then Translation = $d_2 = (-4, 35)$

Conclusion: The order of transformations matters, because the final positions are different.

b) Homogeneous line

i) Give the formula for presenting a homogeneous coordinate as a line. Plot the homogeneous lines l_1 and l_2 in a 2D coordinate system where $l_1 = (1, 0, 1)$ and $l_2 = (6, 0, 2)$. Describe what the lines look like.

ii) Calculate the point where the homogeneous lines l_1 and l_2 join. Are you able to plot this point in a 2D plot? Explain why if not.

Hint:

- the (3x1) homogenous point vector x must satisfy both $I_1^T x = 0$ and $I_2^T x = 0$. In other words, it should be orthogonal to both I_1 and I_2.

Answer:

i)

- A homogeneous coordinate $l = (a, b, c)$ represents a line in the plane through the equation $ax + by + c = 0$
- $l_1 = (1, 0, 1)$ so $x = -1$, $l_2 = (6, 0, 2)$ so $x = -1/3$
- Both lines are vertical and so they are parallel to each other

ii)

- the (3x1) homogenous point vector x should be orthogonal to both I_1 and I_2 and is so calculated using the cross product:

$$l_1 \times l_2 = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 1 & 0 & 1 \\ 6 & 0 & 2 \end{vmatrix}$$

$$x = (0 \cdot 2 - 1 \cdot 0,\ 1 \cdot 6 - 1 \cdot 2,\ 1 \cdot 0 - 0 \cdot 6) = (0, 4, 0)$$

- The intersection point cannot be plotted as a finite point in a 2D graph:
  In homogeneous coordinates, $x = (X, Y, Z)^T$ corresponds to the point $(x, y) = \left(\frac{X}{Z}, \frac{Y}{Z}\right)$, but here $Z = 0$ and so we are not able to plot this point in a 2D plot.

Ex 1.2 OpenCV basics and image manipulation with Python (3 points)

In this exercise we'll start learning how to use Python and OpenCV for computer vision tasks.

In [13]:
```
# Import Python packages needed in this week exercises.
from matplotlib import pyplot as plt
import numpy as np
import cv2
```

1.2 a)

- Load the image located at 'images/lapland.jpeg'.
- State the shape (dimensions) of the image.
- Display the original image and each of its three color channels (Blue, Green, and Red) separately.

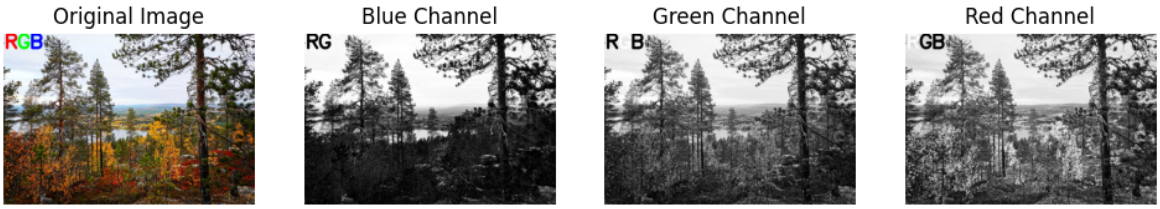
Hint:

- By default, OpenCV's `imread` function reads images in BGR format, while Matplotlib displays images in RGB format.
  - You can use OpenCV's `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` to convert the image from BGR to RGB.
- Images containing a single color channel are typically displayed using a grayscale colormap.
  - In matplotlib, you can use the parameter `cmap='gray'` to achieve this.

In [14]:
```
# Load the image
img = cv2.imread('images/lapland.jpeg')

#shape (dimensions) of the image
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
b_channel, g_channel, r_channel = cv2.split(img)
height, width, channels = img.shape
print(f"shape: height = {height}, width = {width}, Channels = {channels}")

# Display the original image and the color channels
plt.figure(figsize=(12, 6))

# Original image
plt.subplot(1, 4, 1)
plt.imshow(img_rgb)
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 4, 2)
plt.imshow(b_channel, cmap='gray')
plt.title('Blue Channel')
plt.axis('off')

plt.subplot(1, 4, 3)
plt.imshow(g_channel, cmap='gray')
plt.title('Green Channel')
plt.axis('off')

plt.subplot(1, 4, 4)
```

```
plt.imshow(r_channel, cmap='gray')
plt.title('Red Channel')
plt.axis('off')
plt.show()
```

```
shape: height = 1356, width = 2000, Channels = 3
```



1.2 b)

- Load the grayscale image 'images/white_cross.png'.
- Identify the point (x, y) where the white lines intersect. Assume the origin (0,0) is at the top-left corner.

The image below illustrates the coordinate system employed in this exercise, which is also frequently utilized in computer vision.

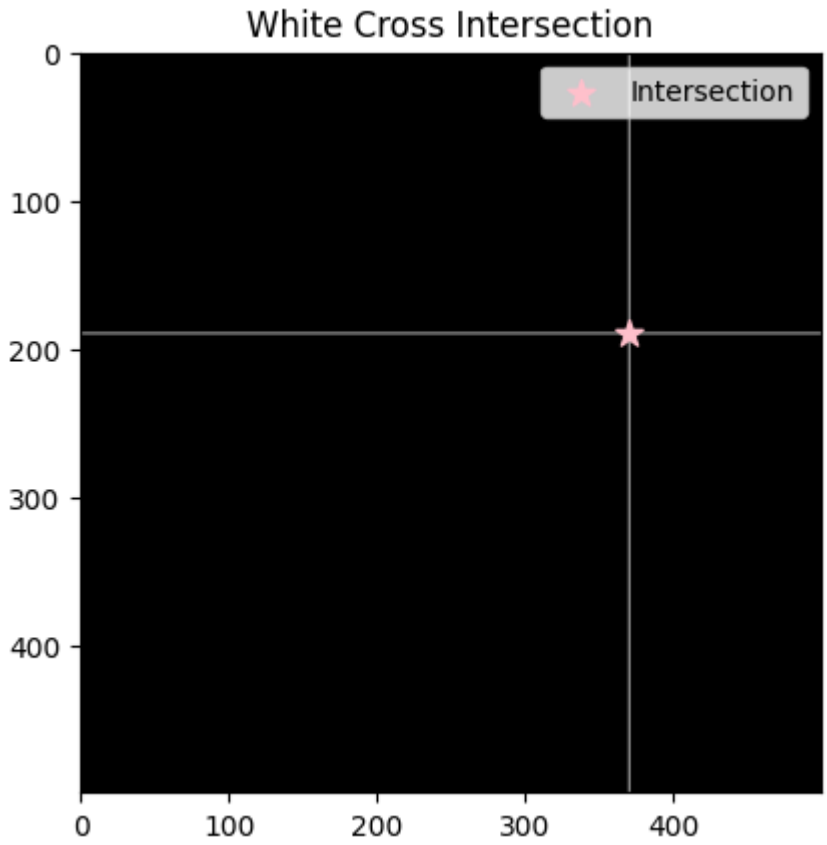
No description has been provided for this image

Hints:

- The image contains only two color values: 0 and 255, indicating it's a 1-channel black and white image.
- The intersecting lines are straight, parallel to the axis, and have a thickness of one pixel.

In [15]:
```
# Load image
img = cv2.imread("images/white_cross.png", cv2.IMREAD_GRAYSCALE)
#white pixels
ys, xs = np.where(img == 255)

# vertical line
x_line = int(np.bincount(xs).argmax())

#horizontal line
y_line = int(np.bincount(ys).argmax())

print(f"Intersection point: ({x_line}, {y_line})")

plt.imshow(img, cmap="gray")
plt.scatter(x_line, y_line, color="pink", marker="*", s=100, label="Intersection
plt.legend()
plt.title("White Cross Intersection")
plt.show()
```

```
Intersection point: (370, 190)
```



1.2 c)

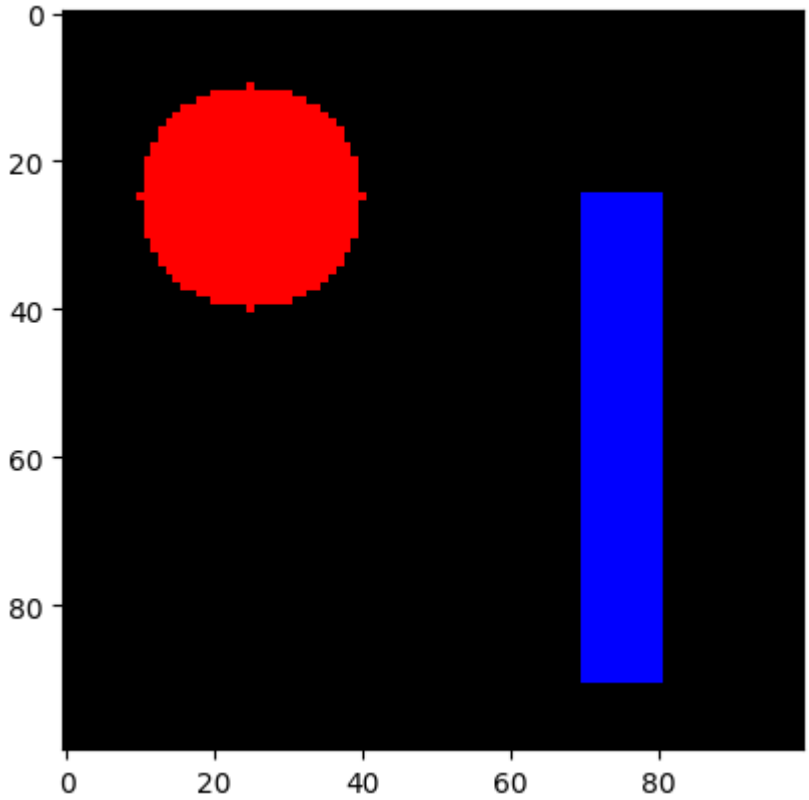
- Load the color image 'images/red_and_blue.jpg'.
- Determine the size of the red circle and blue rectangle in terms of pixel count.

Hint:

- The image only includes RGB color values: (0,0,0), (0,0,255), and (255,0,0).

In [16]:
```
#Load the image
img = cv2.imread("images/red_and_blue.png")
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

red_pixels = (rgb[:, :, 0] == 255) & (rgb[:, :, 1] == 0) & (rgb[:, :, 2] == 0)
blue_pixels = (rgb[:, :, 0] == 0) & (rgb[:, :, 1] == 0) & (rgb[:, :, 2] == 255)

#Count the pixels
num_red = np.count_nonzero(red_pixels)
num_blue = np.count_nonzero(blue_pixels)

#results
print("Red circle pixel count:", num_red)
print("Blue rectangle pixel count:", num_blue)

plt.imshow(rgb)
plt.show()
```

```
Red circle pixel count: 709
Blue rectangle pixel count: 726
```



Ex 1.3 Singular Value Decomposition (SVD) (3 points)

Singular Value Decomposition (SVD): Every $m \times p$ matrix factors into
$A_{n\times p} = U_{n\times n}\Sigma_{n\times p}V^T_{p\times p}$

Singular Value Decomposition can be used for image compression by first applying SVD to the image matrix and then recomposing it by using only some of the singular values contained in matrix $\Sigma_{n\times p}$ and dropping the respective columns and rows in matrices $U_{n\times n}$ and $V^T_{p\times p}$ in order to make the matrices compatible for recomposing the image matrix.

Try this for yourself by decomposing images/lenna.png (open the image as grayscale) with SVD, and then recompose the image by using only the first 50 singular values (you can use the ready-made SVD function in numpy.linalg library, no need to implement it from scratch unless you want to). Also try using different number of singular values. How many singular values are needed before the image no longer appears blurry (Optional extra: how is the image size changed, i.e how much memory you save by compressing the image)?

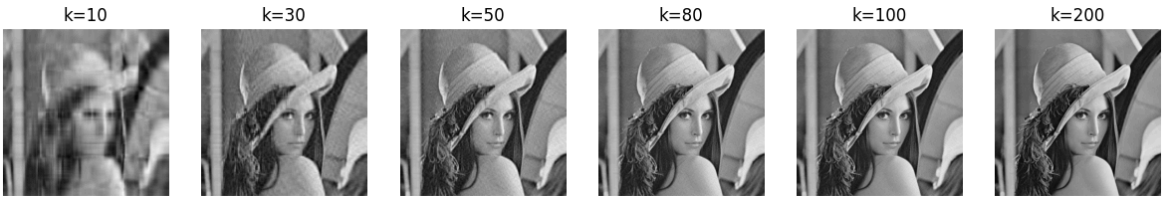
Explain why this method of image compression works, what is the role of singular values and what information they contain? Brief explanation is enough for this assignment.

Answer

- This method works because most of the important information in an image is stored in the largest singular values.

- The singular values in the matrix Σ show how much importance each part of the image has. Large singular values capture the main structures, edges, and shapes of the image, while small singular values contain fine details or noise. By keeping only the largest singular values and removing the small ones, we can compress the image and still keep it looking almost the same. (As in this example, starting from about $k = 80$ the image already looks clear and no longer blurry, which means that the first 80 singular values are enough to represent the important information of the image.)

In [17]:
```
img = cv2.imread("images/lenna.png", cv2.IMREAD_GRAYSCALE)

#SVD
U, S, Vt = np.linalg.svd(img, full_matrices=False)

#reconstruct the image using k singular values
def reconstruct_mg(U, S, Vt, k):
    return np.dot(U[:, :k], np.dot(np.diag(S[:k]), Vt[:k, :]))

# k values (
k_values = [10, 30, 50, 80, 100, 200]
min_dim = min(img.shape)
k_values = [min(k, min_dim) for k in k_values]

# Plot
plt.figure(figsize=(15,5))
for i, k in enumerate(k_values):
    plt.subplot(1, len(k_values), i+1)
    plt.imshow(reconstruct_mg(U, S, Vt, k), cmap='gray')
    plt.title(f'k={k}')
    plt.axis('off')
plt.show()

# Memory calculation for k=50
k = min(50, min_dim)  # ensure k does not exceed image dimensions
compressed_size = k * (U.shape[0] + Vt.shape[1] + 1)
original_size = img.size
print(f"Original size: {original_size} pixels")
print(f"Compressed size with k={k}: {compressed_size} pixels")
print(f"Memory saved: {100*(1 - compressed_size/original_size):.2f}%")
```




```
Original size: 262144 pixels
Compressed size with k=50: 51250 pixels
Memory saved: 80.45%
```